

Información General

- Grupos de máximo 3 estudiantes
- Lenguaje C/C++
- **Fecha de entrega:** Miércoles 25 de septiembre a las 23:59
- **Formato de entrega:** Repositorio de github con código, Makefile, README.md y reporte

Contexto:

En este laboratorio implementarán un simulador de gestión de memoria virtual basado en paginación. Entenderán cómo funciona la traducción VA -> PA, la estructura de tablas de páginas, fallos de página, y cómo diferentes políticas de reemplazo impactan el rendimiento. Cada grupo implementará una política de reemplazo diferente (FIFO o LRU), permitiendo comparación de resultados y análisis de trade-offs.

Requisitos funcionales:

1. Estructura de Memoria Virtual
 - a. Sistema de paginación con tablas de 2 niveles (como x86-64 simplificado)
 - b. Tamaño de página: 4KB (configurable)
 - c. Espacio virtual: 32 bits máximo.
 - d. Espacio físico: configurable, mínimo 256KB
2. Tabla de Páginas de Dos Niveles
 - a. Dirección virtual de 32 bits.

PT1 (10b)	PT2 (10b)	Offset(12b)
-----------	-----------	-------------
 - b.
 - c. PT1 (bits 31-22): índice en tabla nivel 1 (1024 entradas)
 - d. PT2 (bits 21-12): índice en tabla nivel 2 (1024 entradas)
 - e. Offset (bits 11-0): desplazamiento dentro página 4KB
3. Traducción de Direcciones
 - a. Función de traducción VA→PA
 - b. PTE contiene: # página física, bits valid, accessed, dirty
 - c. Fallo de página si la página no está en memoria
 - d. Crear tabla nivel 2 dinámicamente si no existe
4. Manejo de Fallos de Página
 - a. Asignar página física disponible
 - b. Si no hay: aplicar política de reemplazo (FIFO o LRU)
 - c. Registrar estadísticas: # fallos, # reemplazos, tiempo
5. Política de Reemplazo (por grupo)
 - a. FIFO: reemplazar página más antigua (simple, determinista)
 - b. LRU: reemplazar página no usada más recientemente (complejo, mejor hit rate)

Entrada y Salida

El programa acepta archivo de entrada:

```
alloc <bytes> write <virtual_addr> <value> read <virtual_addr> free <virtual_addr>
```

Ejemplo:

```
alloc 8192 write 0 42 write 4096 99 read 0 read 4096
```

Salida (estadísticas finales):

- Total de accesos: N
- Total fallos de página: M
- Hit rate: $(N-M)/N * 100\%$

- Total reemplazos: K
- Política: FIFO o LRU

Requisitos Técnicos

- Compilar sin warnings: gcc -Wall -Werror -std=c99
- Makefile con targets: all, clean, run
- Código comentado y bien estructurado
- Funciones separadas: traducción, fallos, reemplazo
- Sin memory leaks (valgrind)
- Entregables
- Código fuente (.c/.cpp y .h/.hpp/.hcc)
- Makefile funcional
- README.md con instrucciones de compilación y uso
- Reporte de análisis (PDF o markdown) con:
 - Descripción de estructuras de datos
 - Explicación de política de reemplazo
 - Resultados de 2-3 programas de prueba
 - Análisis: cambio de hit rate, # reemplazos
 - Comparación teórica con otra política

Rúbrica de Evaluación

- Funcionalidad Core (35%): Traducción VA→PA correcta, manejo de fallos, tabla 2 niveles funcionando
- Política de Reemplazo (25%): Implementación correcta de FIFO o LRU según asignación
- Calidad de Código (15%): Estructurado, comentado, sin memory leaks, sin warnings
- Reporte de Análisis (15%): Explicación clara, pruebas concretas, análisis crítico
- Bonus: Optimizaciones (+10%): políticas adicionales, visualización, stress testing

Recursos

- OSTEP: Capítulos 18 (Paging), 19 (TLB), 20 (Advanced Page Tables)
- C Reference: man pages para struct, malloc, memcpy
- Debugging: valgrind para memory leaks